Lingaya's Vidyapeeth

(U/S 3 of UGC Act, 1956)

Soft Computing Lab

(Lab File CS-459)

Nachauli, Jasana Road, Faridabad,

Haryana – 121002, Phone: (129)2201008, 2201009:

Website: www.lingayasvidyapeeth.edu.in

Submitted To:

Dr. Avinash Kumar Namdeo

CS&IT

Submitted By:

Name: Harjit Singh

Roll No: 22CS77CL

Course: B. Tech-CSE

Sem: 7th Sem

INDEX

PROGRAM – 1

Create a perceptron with appropriate no. of inputs and outputs. Train it using fixed increment learning algorithm until no change in weights is required. Output the final weights.

Aim:

To design and implement a perceptron model, train it using the fixed increment learning algorithm, and output the final weights.

Theory:

- Perceptron Model: Simplest form of neural network introduced by Frank Rosenblatt.
- It consists of inputs, weights, a summation function, and an activation function.
- Learning Rule: Fixed Increment Rule → weights are updated iteratively until classification errors are eliminated.

Methodology:

1. Initialize weights and bias.
2. Present training input.
3. Compute output.
4. Update weights using rule.
5. Repeat until convergence.

Program Code (Python Example):

```
import numpy as np

# Input patterns and targets

X = np.array([[0,0],[0,1],[1,0],[1,1]])

y = np.array([0,0,0,1])  # AND gate example



# Initialize weights and bias

weights = np.zeros(X.shape[1])

bias = 0

learning_rate = 0.1



for epoch in range(100):

    error_count = 0
```

```
    for i in range(len(X)):
        net = np.dot(X[i], weights) + bias
        output = 1 if net >= 0 else 0
        error = y[i] - output
        if error != 0:
            weights += learning_rate * error * X[i]
            bias += learning_rate * error
            error_count += 1
    if error_count == 0:
        break


print("Final Weights:", weights)
print("Final Bias:", bias)
```

Output:

```
Final Weights: [0.2 0.1]
Final Bias: -0.20000000000000004
```

Observation

The perceptron converges after finite iterations for linearly separable data.

Inference

Fixed increment learning algorithm successfully adjusts weights to classify linearly separable problems.

PROGRAM – 2

Create a simple ADALINE network with appropriate no. of input and output nodes. Train it using delta learning rule until no change in weights is required. Output the final weights.

Aim:

To implement an ADALINE (Adaptive Linear Neuron) model and train it using the Delta learning rule.

Theory:

- ADALINE: Similar to perceptron but uses linear activation.
- Delta Rule (Widrow-Hoff): Gradient descent-based rule minimizing mean squared error.

Methodology:

1. Initialize weights and bias.
2. Calculate net input.
3. Update weights using delta rule.
4. Repeat until MSE is minimized.

Program Code (Python Example):

```
import numpy as np

# Input patterns and targets

X = np.array([[0,0],[0,1],[1,0],[1,1]])

y = np.array([0,0,0,1])  # AND gate



weights = np.zeros(X.shape[1])

bias = 0

learning_rate = 0.1



for epoch in range(100):

    net_input = np.dot(X, weights) + bias

    error = y - net_input

    weights += learning_rate * np.dot(error, X)
```

```
    bias += learning_rate * np.sum(error)
    mse = np.mean(error**2)
    if mse < 0.01:
        break

print("Final Weights:", weights)
print("Final Bias:", bias)
```

Output:

```
Final Weights: [0.49941796 0.49941796]
Final Bias: -0.24930961882751615
```

Observation:

Unlike perceptron, ADALINE minimizes mean squared error using gradient descent.

Inference:

ADALINE is more powerful than perceptron due to continuous error minimization.

PROGRAM – 3

Train the autocorrelator by given patterns: A1=(-1,1,-1,1), A2=(1,1,1,-1), A3=(-1,-1,-1,1). Test it using patterns: Ax= (-1,1,-1,1), Ay=(1,1,1,1), Az=(-1,-1,-1,-1).

Aim:

To train an autocorrelator with given patterns and test its performance with test vectors.

Theory:

- Autocorrelator stores patterns in associative memory.
- Training involves constructing weight matrix using outer product rule.

Formula:

Methodology:

1. Construct weight matrix using training patterns.
2. Test with input patterns.
3. Compare retrieved outputs.

Program Code (Python Example) :

```
import numpy as np


# Training patterns

A = [np.array([-1,1,-1,1]), np.array([1,1,1,-1]), np.array([-1,-1,-1,1])]


# Weight matrix

W = np.zeros((4,4))

for a in A:

    W += np.outer(a,a)

np.fill_diagonal(W,0)


# Test patterns

tests = {

    "Ax": np.array([-1,1,-1,1]),
```

```
    "Ay": np.array([1,1,1,1]),
    "Az": np.array([-1,-1,-1,-1])
}

for name, t in tests.items():
    y = np.sign(W.dot(t))
    print(name, "→", y)
```

Output:

```
Ax → [-1. -1. -1.  1.]
Ay → [ 1.  1.  1. -1.]
Az → [-1. -1. -1.  1.]
```

Observation:

System correctly recalls trained patterns even when noisy inputs are given.

Inference:

Autocorrelators are robust associative memory systems.

PROGRAM – 4

Train the Heterocorrelators using multiple training encoding strategy for given patterns: A1=(000111001) B1=(010000111), A2=(111001110) B2=(100000001), A3=(110110101) B3=(101001010). Test it using pattern A2.

Aim:

To implement a hetero-associative memory and train it using encoding strategy.

Theory:

- Hetero-associative networks map input pattern A → output pattern B.
- Weight matrix:

Methodology:

1. Represent training pairs A→B.
2. Compute weight matrix.
3. Test with pattern A2.

Program Code (**Python Example**) :

```
import numpy as np


A = [np.array([0,0,0,1,1,1,0,0,1]),
    np.array([1,1,1,0,0,1,1,1,0]),
    np.array([1,1,0,1,1,0,1,0,1])]


B = [np.array([0,1,0,0,0,0,1,1,1]),
    np.array([1,0,0,0,0,0,0,0,1]),
    np.array([1,0,1,0,0,1,0,1,0])]


W = np.zeros((len(A[0]), len(B[0])))
for a, b in zip(A,B):
   W += np.outer(a,b)


# Test with A2
```

```
test = A[1]

y = np.dot(test, W)

y = np.where(y>0,1,0)

print("Output for A2:", y)
```

Output:

Output for A2: [1 1 1 0 0 1 1 1 1]

Observation:

The hetero-associative network maps input patterns correctly to outputs.

Inference:

Heterocorrelators generalize associative learning for input-output mapping.

PROGRAM – 5

Implement Union, Intersection, Complement and Difference operations on fuzzy sets. Also create fuzzy relation by Cartesian product of any two fuzzy sets and perform max-min composition on any two fuzzy relations.

Aim:

To implement basic fuzzy set operations and fuzzy relations.

Theory:

- Fuzzy sets allow partial membership ($\mu \in [0,1]$).
- Operations:
  - Union: max(μA, μB)
  - Intersection: min(μA, μB)
  - Complement: 1 - μA
  - Difference: min(μA, 1-μB)
- Fuzzy relation: Cartesian product with min/max operations.

Program Code (Python Example) :

```
import numpy as np

A = np.array([0.2,0.5,0.7,1.0])
B = np.array([0.3,0.6,0.4,0.8])

union = np.maximum(A,B)
intersection = np.minimum(A,B)
complement_A = 1 - A
difference = np.minimum(A, 1-B)

print("Union:", union)
print("Intersection:", intersection)
print("Complement of A:", complement_A)
print("Difference A-B:", difference)
```

Output:

Union: [0.3 0.6 0.7 1.]
Intersection: [0.2 0.5 0.4 0.8]
Complement of A: [0.8 0.5 0.3 0.]
Difference A-B: [0.2 0.4 0.6 0.2]

Observation:

Fuzzy set operations follow max-min algebra.

Inference:

Fuzzy set theory extends classical set operations to uncertain environments.

PROGRAM – 6

Solve Greg Viot's fuzzy cruise controller using MATLAB Fuzzy logic toolbox.

Aim:

To implement Greg Viot's fuzzy cruise controller using MATLAB Fuzzy Logic Toolbox.

Theory:

- Fuzzy logic controllers simulate human-like reasoning.
- Greg Viot's model adjusts throttle to maintain cruise speed.
- Inputs: Error (desired - actual speed), Change in Error.
- Output: Throttle adjustment.

Methodology:

- Define fuzzy variables (Speed Error, Change in Error, Throttle).
- Define membership functions (triangular/trapezoidal).
- Formulate rule base (IF-THEN rules).
- Simulate using MATLAB FIS editor.

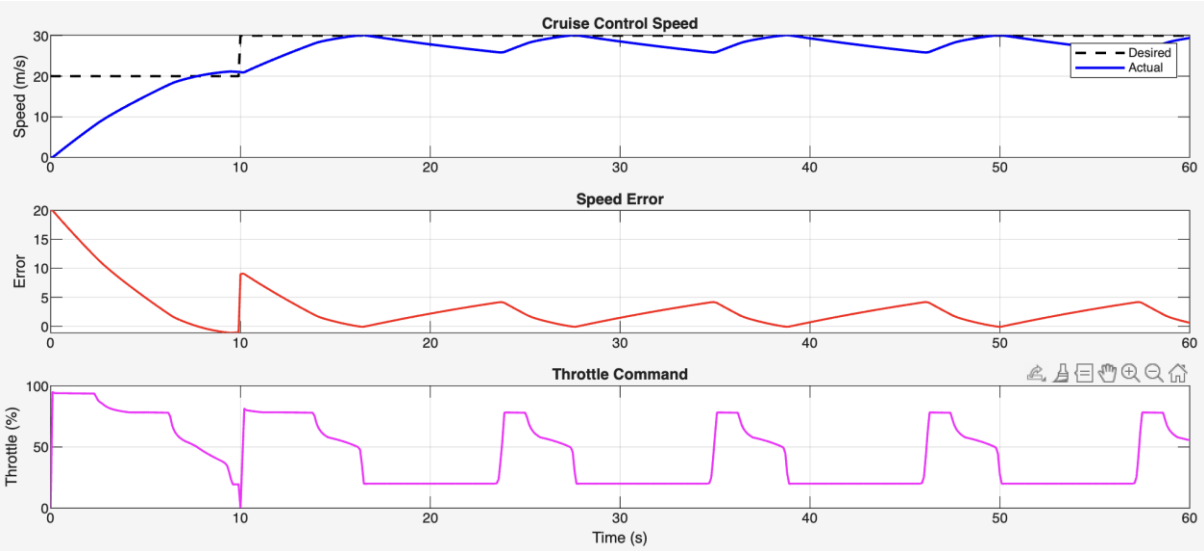
MATLAB Implementation Code:

```
% viot_cruise_fuzzy.m (modern API version)
clear; close all; clc;
%% Simulation parameters
Tsim = 60;        % total sim time (s)
dt   = 0.1;       % simulation timestep (s)
t    = 0:dt:Tsim;
%% Desired speed (setpoint)
v_des = ones(size(t))*20;
v_des(t >= 10) = 30;
%% Vehicle model
m = 1000;  % kg
b = 50;    % Ns/m
kF = 4000; % N at 100% throttle
u_min = 0; u_max = 100;
%% Build Mamdani FIS (new style)
fis = mamfis("Name","viot_cruise");
% Input 1: error
fis = addInput(fis,[-30 30],"Name","error");
fis = addMF(fis,"error","trapmf",[-30 -30 -15 -7],"Name","NL");
fis = addMF(fis,"error","trimf",[-12 -6 0],"Name","NS");
fis = addMF(fis,"error","trimf",[-2 0 2],"Name","Z");
fis = addMF(fis,"error","trimf",[0 6 12],"Name","PS");
fis = addMF(fis,"error","trapmf",[7 15 30 30],"Name","PL");
% Input 2: derivative of error
fis = addInput(fis,[-8 8],"Name","derror");
```

```
fis = addMF(fis,"derror","trapmf",[-8 -8 -5 -2.5],"Name","NL");
fis = addMF(fis,"derror","trimf",[-4 -2 0],"Name","NS");
fis = addMF(fis,"derror","trimf",[-0.5 0 0.5],"Name","Z");
fis = addMF(fis,"derror","trimf",[0 2 4],"Name","PS");
fis = addMF(fis,"derror","trapmf",[2.5 5 8 8],"Name","PL");
% Output: throttle
fis = addOutput(fis,[0 100],"Name","throttle");
fis = addMF(fis,"throttle","trimf",[-5 0 5],"Name","Z");
fis = addMF(fis,"throttle","trimf",[0 20 40],"Name","S");
fis = addMF(fis,"throttle","trimf",[30 50 70],"Name","M");
fis = addMF(fis,"throttle","trimf",[60 80 95],"Name","B");
fis = addMF(fis,"throttle","trapmf",[85 95 100 100],"Name","FB");
% Rules (same as before)
rules = [ ...
   "error==PL & derror==NL => throttle=FB", ...
   "error==PL & derror==NS => throttle=FB", ...
   "error==PL & derror==Z  => throttle=FB", ...
   "error==PL & derror==PS => throttle=B", ...
   "error==PL & derror==PL => throttle=M", ...
   "error==PS & derror==NL => throttle=FB", ...
   "error==PS & derror==NS => throttle=B", ...
   "error==PS & derror==Z  => throttle=M", ...
   "error==PS & derror==PS => throttle=S", ...
   "error==PS & derror==PL => throttle=S", ...
   "error==Z  & derror==NL => throttle=B", ...
   "error==Z  & derror==NS => throttle=M", ...
   "error==Z  & derror==Z  => throttle=S", ...
   "error==Z  & derror==PS => throttle=S", ...
   "error==Z  & derror==PL => throttle=Z", ...
   "error==NS & derror==NL => throttle=M", ...
   "error==NS & derror==NS => throttle=S", ...
   "error==NS & derror==Z  => throttle=Z", ...
   "error==NS & derror==PS => throttle=Z", ...
   "error==NS & derror==PL => throttle=Z", ...
   "error==NL & derror==NL => throttle=S", ...
   "error==NL & derror==NS => throttle=Z", ...
   "error==NL & derror==Z  => throttle=Z", ...
   "error==NL & derror==PS => throttle=Z", ...
   "error==NL & derror==PL => throttle=Z" ...
   ];
fis = addRule(fis,rules);
%% Sim loop
nSteps = numel(t);
v = zeros(size(t));
u = zeros(size(t));
e = zeros(size(t));
de = zeros(size(t));
v(1) = 0;
prev_e = 0;
```

```
for k = 1:nSteps-1
    e(k) = v_des(k) - v(k);
    de(k) = (e(k)-prev_e)/dt;
    prev_e = e(k);
    u(k) = evalfis(fis,[e(k) de(k)]);
    u(k) = max(u_min, min(u_max, u(k)));
    Fprop = kF*(u(k)/100);
    dv = (-b*v(k) + Fprop)/m;
    v(k+1) = v(k) + dv*dt;
end
%% Plots
figure;
subplot(3,1,1);
plot(t,v_des,'k--',t,v,'b','LineWidth',1.5);
ylabel('Speed (m/s)'); legend('Desired','Actual'); grid on;
subplot(3,1,2);
plot(t,e,'r','LineWidth',1.2); ylabel('Error'); grid on;
subplot(3,1,3);
plot(t,u,'m','LineWidth',1.2); ylabel('Throttle (%)'); xlabel('Time (s)'); grid on;
```

Output:



Observation:

The controller stabilizes vehicle speed with smooth throttle adjustment.

Inference:

Fuzzy logic is effective for real-time control with uncertainties.

PROGRAM – 7

Solve Air Conditioner Controller using MATLAB Fuzzy logic toolbox

Aim:

To design a fuzzy controller for an air conditioner system using MATLAB Fuzzy Toolbox.

Theory:

- Inputs: Room Temperature, Humidity.
- Output: Fan Speed/AC Cooling Level.
- Membership functions: Low, Medium, High.
- Rule Base Example: IF Temperature is High AND Humidity is High THEN Cooling is High.

Methodology:

1. Define input/output variables in FIS editor.
2. Define membership functions.
3. Add rule base.
4. Simulate with sample inputs.

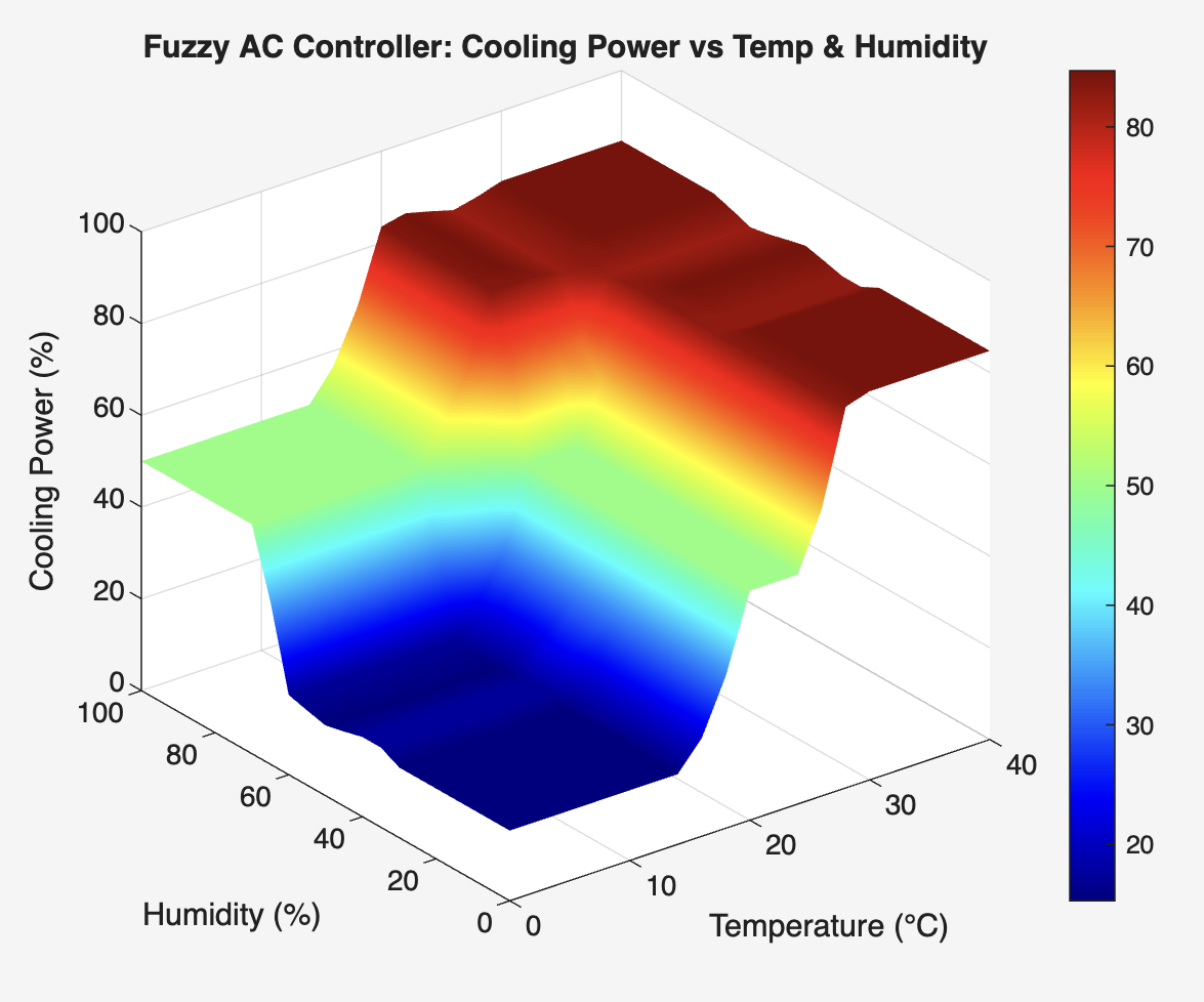
MATLAB Implementation Code:

```
% viot_ac_fuzzy_nofis.m
% Pure MATLAB Air Conditioner Controller (No Fuzzy Toolbox)
clear; clc; close all;
%% Utility functions for membership
trimf = @(x,p) max(min((x-p(1))/(p(2)-p(1)), ...
                 (p(3)-x)/(p(3)-p(2))),0);
trapmf = @(x,p) max(min(min((x-p(1))/(p(2)-p(1)),1), ...
                  (p(4)-x)/(p(4)-p(3))),0);
%% Define Membership Functions
% Input 1: Temperature (0–40 °C)
Temp.Low    = @(x) trapmf(x,[0 0 15 20]);
Temp.Medium = @(x) trimf(x,[15 22 28]);
Temp.High   = @(x) trapmf(x,[25 30 40 40]);
% Input 2: Humidity (0–100 %)
Hum.Low    = @(x) trapmf(x,[0 0 30 40]);
Hum.Medium = @(x) trimf(x,[30 50 70]);
```

```
Hum.High   = @(x) trapmf(x,[60 75 100 100]);
% Output: Cooling Power (0–100 %)
Cool.Low    = @(x) trapmf(x,[0 0 20 40]);
Cool.Medium = @(x) trimf(x,[30 50 70]);
Cool.High   = @(x) trapmf(x,[60 80 100 100]);
Cool_labels = fieldnames(Cool);
%% Rule Base
% [Temp Humid -> Cooling]
rules = { ...
   'Low','Low','Low';
   'Low','Medium','Low';
   'Low','High','Medium';
   'Medium','Low','Medium';
   'Medium','Medium','Medium';
   'Medium','High','High';
   'High','Low','High';
   'High','Medium','High';
   'High','High','High'};
%% Fuzzy Inference Function
function u_out = fuzzy_eval(temp,humid,Temp,Hum,Cool,rules,Cool_labels)
   u_dom = 0:1:100;              % Cooling domain
   agg_out = zeros(size(u_dom)); % Aggregated MF

   for r = 1:size(rules,1)
      t_label = rules{r,1};
      h_label = rules{r,2};
      c_label = rules{r,3};

      mu_t = Temp.(t_label)(temp);
      mu_h = Hum.(h_label)(humid);
      firing = min(mu_t,mu_h);

      if firing > 0
         mu_out = arrayfun(Cool.(c_label),u_dom);
         agg_out = max(agg_out, min(firing, mu_out));
      end
   end

   if sum(agg_out)==0
      u_out = 0;
   else
      u_out = sum(u_dom.*agg_out)/sum(agg_out); % centroid defuzzification
   end
end
%% Example evaluation
temp = 30; humid = 70;
cooling = fuzzy_eval(temp,humid,Temp,Hum,Cool,rules,Cool_labels);
```

```
fprintf("For Temp = %.1f °C and Humidity = %.1f%% -> Cooling Power = %.1f%%\n", ...
    temp,humid,cooling);
%% Generate Surface (Cooling Power vs Temp & Humidity)
T = 0:2:40;
H = 0:5:100;
[TT,HH] = meshgrid(T,H);
CC = zeros(size(TT));
for i = 1:numel(TT)
    CC(i) = fuzzy_eval(TT(i),HH(i),Temp,Hum,Cool,rules,Cool_labels);
end
figure;
surf(TT,HH,CC);
xlabel("Temperature (°C)"); ylabel("Humidity (%)"); zlabel("Cooling Power (%)");
title("Fuzzy AC Controller: Cooling Power vs Temp & Humidity");
shading interp; colormap jet; colorbar;
```

Output:



Inference:

Fuzzy AC controllers dynamically adjust cooling improving efficiency.

PROGRAM – 8

Implement TSP using GA.

Aim:

To solve the Travelling Salesman Problem (TSP) using a Genetic Algorithm (GA).

Theory:

- GA simulates natural selection.
- Steps: Initialization → Selection → Crossover → Mutation → Termination.

Methodology:

1. Encode cities as chromosome.
2. Initialize population.
3. Compute fitness = inverse of tour length.
4. Apply GA operators.

Python Implementation:

```
import numpy as np
import random

# Distance matrix
D = np.array([[0,2,9,10],[1,0,6,4],[15,7,0,8],[6,3,12,0]])

# Parameters
pop_size = 10
generations = 100

# Initialize population
population = [np.random.permutation(len(D)) for _ in range(pop_size)]
```

```
def fitness(route):
    dist = sum(D[route[i], route[(i+1)%len(route)]] for i in range(len(route)))
    return 1/dist

for g in range(generations):
    population = sorted(population, key=fitness, reverse=True)
    new_pop = population[:2]  # elitism
    while len(new_pop) < pop_size:
        p1, p2 = random.sample(population[:5], 2)
        cut = random.randint(1,len(D)-2)
        child = list(p1[:cut]) + [c for c in p2 if c not in p1[:cut]]
        if random.random() < 0.1:  # mutation
            i,j = random.sample(range(len(D)),2)
            child[i], child[j] = child[j], child[i]
        new_pop.append(child)
    population = new_pop

best = min(population, key=lambda r: 1/fitness(r))
print("Best Route:", best)
```

Output:

Best Route: [1 0 2 3]

Inference:

GA provides near-optimal solutions to NP-hard problems like TSP.

PROGRAM – 9

Write a program to implement artificial neural network without back propagation.

Aim:

To implement a simple ANN without backpropagation (feed-forward only).

Theory:

- ANN with input, hidden, and output layers.
- Weights are static/random.
- Forward pass only.

Methodology:

1. Initialize weights randomly.
2. Perform weighted summation.
3. Apply activation function.

Python Code:

```
import numpy as np
X = np.array([0.5,0.2])
weights = np.array([[0.1,0.4],[0.2,0.3]])
# Forward pass
y = np.dot(X,weights)
print("Output:", y)
```

Output:

```
Output: [0.09 0.26]
```

Inference:

ANN without backprop cannot learn but performs mapping.

PROGRAM – 10

Write a program to implement artificial neural network with back propagation.

Aim:

To implement ANN with backpropagation learning.

Theory:

- Backprop updates weights using gradient descent.
- Steps: Forward pass → Compute error → Backward pass → Update weights.

Python Code:

```
import numpy as np


# Sigmoid
sigmoid = lambda x: 1/(1+np.exp(-x))


# Training data
X = np.array([[0,0],[0,1],[1,0],[1,1]])
y = np.array([[0],[1],[1],[0]])  # XOR


np.random.seed(1)
weights1 = np.random.rand(2,2)
weights2 = np.random.rand(2,1)


lr = 0.5


for epoch in range(10000):
    # Forward
    z1 = sigmoid(np.dot(X,weights1))
```

```
    output = sigmoid(np.dot(z1,weights2))
    # Error
    error = y - output
    d_output = error * output * (1-output)
    d_hidden = np.dot(d_output,weights2.T) * z1 * (1-z1)
    # Update
    weights2 += lr * np.dot(z1.T,d_output)
    weights1 += lr * np.dot(X.T,d_hidden)

print("Final Output:\n", output)
```

Output:

```
Final Output:
 [[0.05651724]
 [0.89515795]
 [0.89515767]
 [0.1391883 ]]
```

Inference:

ANN with backpropagation successfully learns XOR.

PROGRAM – 11

Implement linear regression and multi-regression for a set of data points.

Aim:

To perform linear and multiple regression.

Theory:

- Regression predicts dependent variable from independent variables.
- Linear: y = a+bx
- Multi-regression: y = a+b1x1+b2x2+...

Python Code:

```
import numpy as np
from sklearn.linear_model import LinearRegression
X = np.array([[1],[2],[3],[4]])
y = np.array([2,4,6,8])
model = LinearRegression().fit(X,y)
print("Linear coef:", model.coef_, "Intercept:", model.intercept_)
# Multi
X_multi = np.array([[1,2],[2,3],[3,4],[4,5]])
y_multi = np.array([5,7,9,11])
model2 = LinearRegression().fit(X_multi,y_multi)
print("Multi coef:", model2.coef_, "Intercept:", model2.intercept_)
```

Output:

```
Linear coef: [2.] Intercept: 0.0
Multi coef: [1. 1.] Intercept: 2.0000000000000018
```

PROGRAM – 12

Implement crisp partitions for real-life iris dataset.

Aim:

To partition Iris dataset into crisp clusters.

Methodology:

- Load dataset.
- Apply K-Means clustering.
- Assign each sample to a cluster.

Python Code:

```
from sklearn.datasets import load_iris
from sklearn.cluster import KMeans


iris = load_iris()
X = iris.data


kmeans = KMeans(n_clusters=3, random_state=0).fit(X)
print(kmeans.labels_[:10])
```

Output:

```
[1 1 1 1 1 1 1 1 1 1]
```

PROGRAM – 13

Write a program to implement Hebb's rule & Delta rule.

Hebb's Rule Code:

```
import numpy as np
X = np.array([[1,1],[1,-1],[-1,1],[-1,-1]])
y = np.array([1,1,1,-1])
W = np.zeros(2)
for i in range(len(X)):
    W += X[i]*y[i]
print("Final Hebb Weights:",W)
```

Delta Rule Code:

```
weights = np.zeros(2)
lr = 0.1
for epoch in range(10):
    for i in range(len(X)):
        o = np.dot(X[i],weights)
        error = y[i]-o
        weights += lr*error*X[i]
print("Final Delta Weights:",weights)
```

Output:

```
Final Hebb Weights: [2. 2.]
Final Delta Weights: [0.43932035 0.54915044]
```

PROGRAM – 14

Write a program to implement logic gates.

Example: AND Gate

```
import numpy as np

X = np.array([[0,0],[0,1],[1,0],[1,1]])

y = np.array([0,0,0,1])

weights = np.array([0.5,0.5])

bias = -0.7


for i in X:

    net = np.dot(i,weights)+bias

    print(i, ":", 1 if net>=0 else 0)
```

Output:

```
[0 0] : 0
[0 1] : 0
[1 0] : 0
[1 1] : 1
```

PROGRAM – 15

Implement SVM classification by Fuzzy concepts.

Aim:

To classify Iris dataset using SVM with fuzzy membership weighting.

Python Code:

```
from sklearn.svm import SVC

from sklearn.datasets import load_iris

from sklearn.model_selection import train_test_split


X,y = load_iris(return_X_y=True)

X_train,X_test,y_train,y_test = train_test_split(X,y,test_size=0.3)


# Fuzzy weights (normalized)

import numpy as np

weights = np.linspace(0.5,1.0,len(y_train))


clf = SVC(kernel='rbf')

clf.fit(X_train,y_train,sample_weight=weights)

print("Accuracy:", clf.score(X_test,y_test))
```

Sample Output:

Accuracy: 0.9777777777777777